CSC3002 Final Review

Li Mohan 122090268

C++ Compiled/Interpreted

C++, C... VS Python, Ruby...

Compiler:

C++ program --**compile**-> object code --**linking**-> executable file -> running

一次编译，运行不需要源代码
一般更快

Interpreter:

Python program --**interpreter**-> running

一般可以跨平台 (依赖不同平台的解释器)
运行需要源代码

C++ Program Structure

```cpp
#include <iostream>
#include "CSC3002.h"
#define MAXN 100000

#ifdef NAMESPACE
using namespace std;
#endif
using std::endl;

int main() {
  std::cout << "Hello, world" << endl;
  return 0;
}
```

C++ Variables

Type name = value;

Value, name, address, Type.

Variable name: **letters**, **digits**(!start), **underscore**, **!keywords**

Scope: name valid, visibility

Extent (lifetime): a variable has a meaningful value

::

const

```
#include <iostream>
using namespace std;
int x = 2;
int main() {
    int x = 10;
    {
      int x = 7;
      cout << x << endl;   7
      cout << ::x << endl; 2
    }          显式全局变量调用
    cout << x << endl;   10
    cout << ::x << endl; 2
    return 0;
}
```

C++ Operator

Binary operator(x*y) and **Unary operator (-x)**

Type cast:

14/5, double(14)/5, 14/double(5), double(14/5)

Precedence: change associativity

The value of the comma is the value of the second part, and the comma has the lowest priority.

cat = 12,13; cat = 12

dog = (cat = 12, 13);

13

TABLE 1-4 Operators available in C++

Operators organized into precedence groups	Associativity
`()` `[]` `->` `.`	*left*
unary operators: `-` `++` `--` `!` `&` `*` `~` `(type)` `sizeof`	*right*
`*` `/` `%`	*left*
`+` `-`	*left*
`<<` `>>`	*left*
`<` `<=` `>` `>=`	*left*
`==` `!=`	*left*
`&`	*left*
`^`	*left*
`|`	*left*
`&&`	*left*
`||`	*left*
`?:`	*right*
`=` *op=*	*right*

C++ Precedence Practice

```
if (test_3() || test_1() && test_2()) {
}
```

test3() || (test_1() && test2())

test3() executed (short-circuit)

TABLE 1-4 Operators available in C++

Operators organized into precedence groups	Associativity
() [] -> .	*left*
unary operators: - ++ -- ! & * ~ (*type*) sizeof	*right*
* / %	*left*
+ -	*left*
<< >>	*left*
< <= > >=	*left*
== !=	*left*
&	*left*
^	*left*
|	*left*
&&	*left*
||	*left*
?:	*right*
= *op*=	*right*

C++ Control Statement

Cascading **if** statement

```
if (condition1) {
    statements1
} else if (condition2) {
    statements2
. . . more else/if conditions . . .
} else {
    statementselse
}
```

```
while ( condition ) {
    statements to be repeated
}
```

```
for ( init ; test ; step ) {
    statements to be repeated
}
```

```
condition ? expression1 : expression2
```

```
max = (x > y) ? x : y;
```

```
switch ( expression ) {
  case v1:
    statements to be executed if expression = v1
    break;
  case v2:
    statements to be executed if expression = v2
    break;
  . . . more case clauses if needed . . .
  default:
    statements to be executed if no values match
    break;
}
```

C++ Programming Paradigms

C++ supports the ***procedural*** and ***object-oriented*** paradigms naturally, supports the *functional* paradigm through the **`<functional>`** interface, and supports many other paradigms through various external libraries.

- **Imperative** programming paradigm: an explicit sequence of statements that change a program's state, specifying how to achieve the result.
  - **Structured**: Programs have clean, **`goto`**-free, nested control structures.
  - **Procedural**: Imperative programming with procedures operating on data.
  - **Object-Oriented**: Objects have/combine states/data and behavior/methods; Computation is effected by sending messages to objects (receivers).
    - **Class-based**: Objects get their states and behavior based on membership in a class.
    - **Prototype-based**: Objects get their behavior from a prototype object.

Procedure Oriented
Object-Oriented

C++ Function

Argument: actual values
Parameter: placeholder

Return **bool**: **predicate**
No return (**void**): **procedure**

Function prototype

Advantage:

shorten a program
easier to read
simplify program maintenance
...

```
void test(int x = 3);
int main()
{
}
void test(int x = 3) {
  cout << "test" << endl;
}
```

Default values: only in function prototype, must appear at the end

Overloaded Function Signature: **number types of the argument**
(int x) and (int& x)

Function pointer: Pass the function as a parameter

```
void display(int x, int y) {
    cout << x << " " << y << endl;
}
int main() {
  void (*pf)(int, int) = display;
  (*pf)(2, 3);
  pf(2, 3);
}
```

```
const double * f1(const double *, int);
//f2,f3 are the same
//下面将创建一个存储三个函数的函数指针数组
const double * (*pa[3])(const double *, int) = {f1, f2, f3}; //pa中存储的是指针
```

C++ Character and String

C++/C use ASCII '0':48 'a':97 'A': 65 Space:32 char ch = 'a'; or char ch = 97

C-Style strings:

```
char cstr[10];
char cstr[] = "hello";
Char *cstr = "hello";
char cstr[] = { 'h', 'e', 'l', 'l', 'o', '\0' };
```

'a' != "a"

Concatenation of Strings

- C-Style String can't use "+"
- C-Style String and String can use "+"

```
char name1[] = "Ray";
std::string name2 = "Ray";
std::cout << "Hello, " + name1 << std::endl;
std::cout << "Hello, " + name2 << std::endl;
```

C++ Character and String

Allocate size and **effective size** **(char *c) sizeof(c)**

```
char ch[6] = "hello", ch2[10];
cout << "sizeof(ch2): " << sizeof(ch2) << endl;
cout << "ch is " << ch << endl << "sizeof(ch): " << sizeof(ch) << endl << "strlen(ch): " << strlen(ch) << endl;
ch[5] = ','; ch[6] = 'w'; ch[7] = '\0';
cout << "ch is " << ch << endl << "sizeof(ch): " << sizeof(ch) << endl << "strlen(ch): " << strlen(ch) << endl;
strcpy(ch, "CSC3002 Final Review");
cout << "ch is " << ch << endl << "sizeof(ch): " << sizeof(ch) << endl << "strlen(ch): " << strlen(ch) << endl;
```

C++ Streams

A *stream* is an abstraction (object) that represents an input source or output destination of characters of indefinite length

In c++, no copy/assignment

Insertion operator: << an output stream on the left, an expression of any type on its right

Extraction operator: >> symmetrical

Construct a new ifstream

Call open() method

read data

Call close() method

get() and getline()

EOF and fail indicator

```
int ch;
while ((ch = infile.get()) != EOF) {
    cout.put(ch);
}
```

```
while ( !infile.fail() ) {
    . . . Perform the necessary opera
}
```

<sstream> **istringstream** class read data/output data from/to a string

C++ enum

```
enum enumType {Monday, Tuesday, Wednesday, Thursday, Friday, Saturday, Sunday};
```

Default start value is 0, If an element is not declared, It defaults to be previous one +1

```
enum enumType {red, blue = 2, green, white = 10};
```

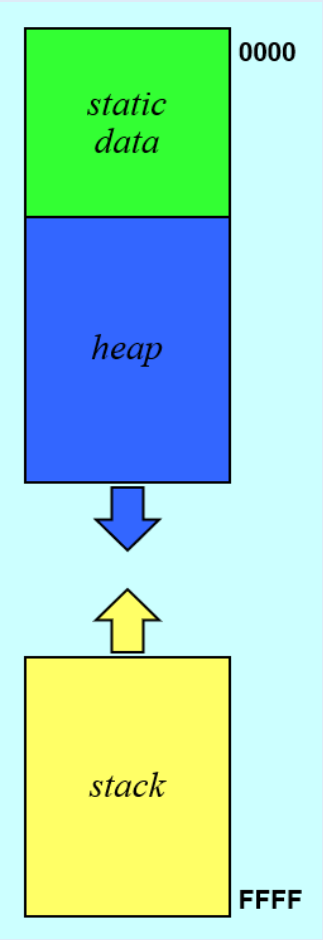
Enumerations **do not have arithmetic operators**, but they do have **assignment operators**

The enumerator is an integer and can be automatically promoted to the int type, but the **int cannot be converted to the enumeration.**

```
enum g{red, blue, org};
g a;
a = red;
a++;
a = blue + red;

int color = blue;
a = 3;
color = 3 + red;
```

C++ Memory management



Program code and global variables/constants

Dynamic allocate memory

Local variables

When you invoke a function, the following actions occur:

1. The calling function evaluates the argument expressions in its own context.
2. C++ then copies each argument value into the corresponding parameter variable, which is allocated in a newly assigned region of memory called a *stack frame*.
3. C++ then evaluates the statements in the function body, using the new stack frame to look up the values of local variables.
4. When C++ encounters a `return` statement, it computes the return value and substitutes that value in place of the call.
5. C++ then discards the stack frame for the called function and returns to the caller, continuing from where it left off.

C++ Pointers and Arrays

1 byte (8 bits)	2 bytes (16 bits)	4 bytes (32 bits)	8 bytes (64 bits)	16 bytes (128 bits)
`char` `bool`	`short`	`int` `float`	`long` `double`	`long double`

`sizeof(t)` returns the actual number of bytes required to store a value of the type `t`;
`sizeof x` returns the actual memory size of the variable `x`.

X must be a variable t can be a type

```
test.cpp:5:17: warning: format specifies type 'int' but the argument has t
ype 'unsigned long' [-Wformat]
        printf("%d\n", sizeof(int));
                ~~     ^~~~~~~~~~
                %lu
test.cpp:6:17: warning: format specifies type 'int' but the argument has t
ype 'unsigned long' [-Wformat]
        printf("%d\n", sizeof(a));
                ~~     ^~~~~~~~~
                %lu
test.cpp:7:17: warning: format specifies type 'int' but the argument has t
ype 'unsigned long' [-Wformat]
        printf("%d\n", sizeof a);
                ~~     ^~~~~~~
                %lu
test.cpp:8:23: error: expected parentheses around type name in sizeof expr
ession
        printf("%d\n", sizeof int);
                               ^
                               (   )
3 warnings and 1 error generated.
```

Lvalue and rvalue
Lvalue: can be on the left side of an assignment(incomplete), can find its address
Lvalue: variable, string literal(&"hello")…
Rvalue: return value, expression…

C++ Pointers

Pointer:

Pointers allow you to refer to a large data structure in a compact way.
Pointers make it possible to reserve new memory during program execution
Pointers can be used to record relationships among data items

The address-of operator &
The value-pointed-to operator * (dereferencing)

Refernece (int& x)

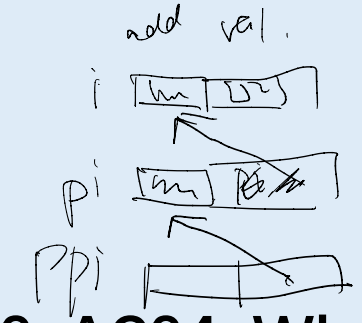
-> Operator

pp->get()
***pp.get() = (*pp).get()**

C++ Pointers Practice

1. True or false: *&x == x, &*p == p

√ X



2. int i = 2023; int *pi = &i; int **ppi = &pi.
Known that address of i and pi are 0xAC00 and 0xAC04. What are the values of ppi, **ppi, *ppi

0xAC04 2023 AC00

3. int* p1, p2; int *p3, p4; p1, p3 pointer
The type of p1, p2, p3, p4 p2, p4 int

4. p, -p, *p, &p which may be an lvalue

左右 呀

C++ Pointers Practice

*(p[10])　(*p)[10]

5. int *p[10];　int (*p)[10];　what are they?

10个指针数组　指向 [10] 数组

6. How to use pointers to represent a[i][j]

m行n列

C++ Arrays

The name of an array is an address

int arr[100]; arr = &arr[0] (numerically)

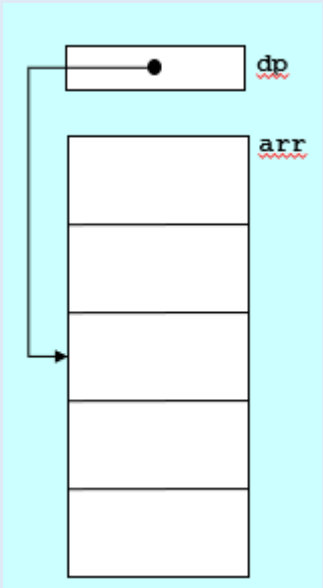
An array is often treated as a pointer (not entirely equivalent).

an array is a non-modifiable lvalue

```
double arr[5];
double * dp = arr;
```

```
arr[i] ≡ *(arr+i) ≡ *(dp+i) ≡ dp[i]
&arr[i] ≡ arr+i ≡ dp+i ≡ &dp[i]
```

i元素地址



C++ Arrays

The name of an array is an address

int arr[100]; arr = &arr[0] (numerically)

```
int a[] = {0, 1, 2, 3};
int * p = a; // &a[0]
```

	a	**&a**	**&a[0]**	**p**
Type	array or *used as* pointer to an **int**	address of an array	address of an **int**	pointer to an **int**
Size of	16 (4***int** length)	4 (a word)	4 (a word)	4 (a word)
Lvalue	Yes (non-modifiable)	No	No	Yes
Value	**ADDRESS1**	**ADDRESS1**	**ADDRESS1**	**ADDRESS1**
*****	**0**	**ADDRESS1**	**0**	**0**
&	**ADDRESS1**	N/A	N/A	**ADDRESS2**
+1	**ADDRESS1** **+1*int** length	**ADDRESS1** **+4*int**	**ADDRESS1** **+1*int**	**ADDRESS1** **+1*int**

C++ C-Style String address

“Hello, world” is also an address (lvalue)

```
cout << &"ciallo" << endl;
```

```
0x5572f1797004
```

char msg[] = “Hello, world”;
char *msg = “Hello, world”;

```
char msg1[] = "ciallo";
char msg2[] = "ciallo";
char *msg3 = "ciallo";
char *msg4 = "ciallo";
cout << msg1 << endl << &"ciallo" << endl << (int*)msg1 << endl << (int*)msg2 << endl
  << (int*)msg3 << endl << (int*)msg4 << endl << (msg1 == msg2) << endl;
```

```
ciallo
0x56240c4cf004
0x7ffc8314b10a
0x7ffc8314b111
0x56240c4cf004
0x56240c4cf004
0
```

char msg[] = “Hello, world”; stack, different
char *msg = “Hello, world”; static data, same

C++ Arrays Practice

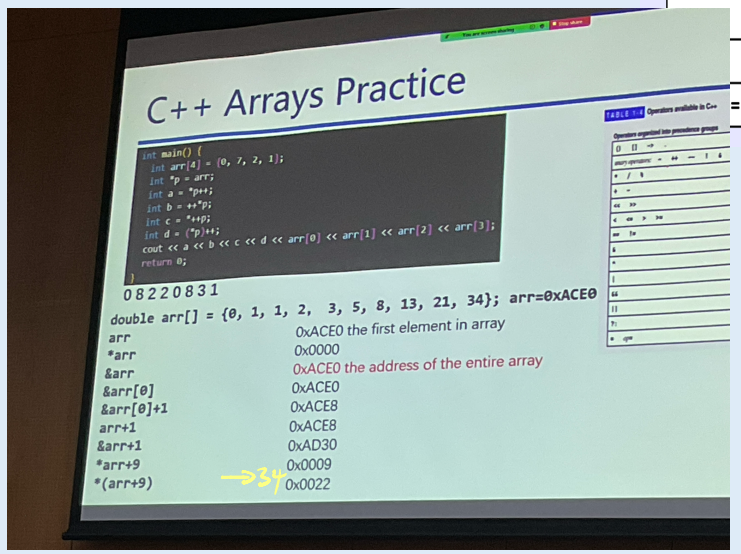
```cpp
int main() {
  int arr[4] = {0, 7, 2, 1};
  int *p = arr;
  int a = *p++; *(p++) 先引再动, 指0
  int b = ++*p; *p=7. ++7=8. 先改再取
  int c = *++p; ++p 先动再引 指2
  int d = (*p)++; d不. 指针++. 还是2, 但arr[2]++ ⇒3
  cout << a << b << c << d << arr[0] << arr[1] << arr[2] << arr[3];
  return 0;
}
```

0 8 2 2 0 8 3 1

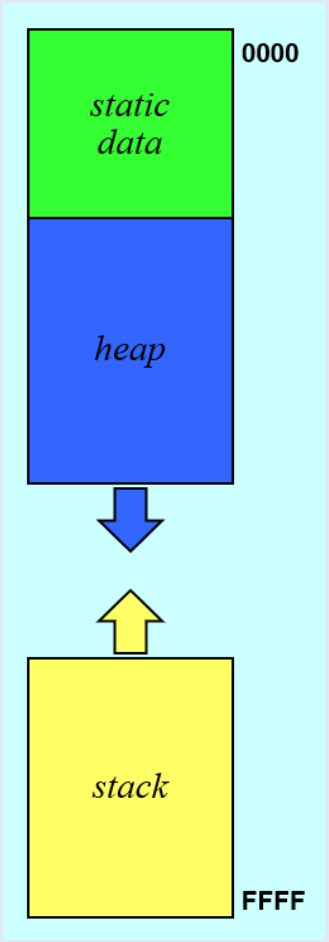
```
double arr[] = {0, 1, 1, 3, 5, 8, 13, 21, 34}; arr=0xACE0
arr
*arr
&arr
&arr[0]
&arr[0]+1
arr+1
&arr+1
*arr+9
*(arr+9)
```

TABLE 1-4 Operators available in C++

Operators organized into precedence groups	Associativity
() [] -> .	*left*
unary operators: - ++ -- ! & * ~ (*type*) sizeof	*right*
* / %	*left*
+ -	*left*
<< >>	*left*
< <= > >=	*left*
== !=	*left*
&	*left*
^	*left*
|	*left*
&&	*left*
	left
	right
=	*right*



C++ Dynamic Memory Allocation




```
int * pi = new int;
```

```
int * arr = new int[10000];
```

```
delete pi;
delete [] arr;
```

Dynamic allocate memory

```
delete pi;
pi = nullptr; //pi = NULL
```

delete ptr only frees the memory space pointed by ptr, but not the memory space occupied by ptr itself

The pointer is dangling

Need to nullify

Local variables

Delete NULL is safe

```
char *p1 = new char;
char *p2 = new char;
p2 = p1;
delete p1;
p1 = NULL;
delete p2;
```

C++ class basic and friend

```cpp
#include <iostream>
#include <string>
using namespace std;
class Point{
 public:
  Point() {
    this->x_ = 0;
    this->y_ = 0;
    this->val_ = 0;
  }
  Point(int x, int y, int val) : x_(x), y_(y), val_(val) {}
  ~Point() {}
  void Display() {
    cout << "(" << x_ << ", " << y_ << ") = " << val_ << endl;
  }
  void Set(int x, int y, int val);
  friend void SetValue(Point& point, int val);
 private:
  int x_;
  int y_;
  int val_;
};
```

constructor

Destructor

Friend function

Instance

```cpp
void Point::Set(int x, int y, int val) {
  x_ = x;
  y_ = y;
  val_ = val;
}

void SetValue(Point& point, int val) {
  point.val_ = val;
}

int main() {
  Point point(1, 2, 3);
  point.Display();
  SetValue(point, 4);
  point.Display();
}
```

Public and Private Struct and class

```cpp
void operator<<(ostream & os, const Stock & t) {
    os << t.hours << " " << t.minutes << endl;
}
```

A friend function of a class is defined outside the class, but has access to all private and protected members of the class.

C++ class operator overload

```
Point operator+(const Point& other) {
  return Point(x_ + other.x_, y_ + other.y_, val_ + other.val_);
}
```

Can we use Point& for the return value type?

```
Point point_1(1, 1, 1);
Point point_2(2, 2, 2);
Point point_3(3, 3, 3);
Point new_point = point_1 + point_2 + point_3;
new_point.Display();
```

New_point = point_1.operator+(point_2)
New_point = point_1 + point_2

If we define a class's * operator as a instance * constant like a = b * 2 (a, b are instances of a class)

a = 2 * b is wrong
So we can add a friend function:
    friend Stock operator*(const & Stock, double m);
Or we can add a Stock(double);

Function object: operator()(parameter list) {}
    can define state variable in class

C++ class Copy

- The operator operator=, which takes care of assignments
- A copy constructor, which takes care of by-value parameters

type **&** *type*`::`**operator=(const** *type* **& src)** This allows you to chain assignments together.

type`::`*type*`(`**const** *type* **& src)**

Shallow copying: default copy behavior

Deep copying: copying one collection to another creates an entirely new copy of the collection

```
int *v;
Useless& operator=(const Useless& other) {
  if (this == &other) {
    return *this;
  }
  v = new int(*(other.v));
}
```

Violate it: put them in private (like streams)

C++ class

default
delete
explicit
const

```
Useless(const Useless & un) = default;
explicit Useless(double x); //only on parameter
void put(double x);
void put(int x) = delete;
void seek() const;
```

C++ const

Constant definitions: const int x = 10;

Constant call by reference: void test(const Stock& stock)

Constant method: guarantees that the method will not change the object
int Stack::size() const {}

Const x = 1 in a class doesn't mean all the instance's x is the same address

```
Vector & Vector::operator+(const Vector &) const { return Vector(x + b.x, y + b.y); }
```

```
net = force1 + force2; //valid
force1 + force2 = net; //valid, return a class object
```

const typeName* variable_name = adress; //*p is const   const pointer
typeName *const variable_name = adress; // p is const   pointer to const
const typeName* const variable_name;

C++ Inheritance

```
class subclass : public superclass {
    body of class definition
};
```

Base class member access specifier	Type of Inheritence		
	Public	Protected	Private
Public	Public	Protected	Private
Protected	Protected	Protected	Private
Private	Not accessible (Hidden)	Not accessible (Hidden)	Not accessible (Hidden)

C++ Inheritance Class Function

Overloading: same name and different signatures

Overriding: same signatures but different implementations in different classes

Redefining: derived class's same name function will hide the base class's function (except same signature && virtual)

```cpp
class A {
 public:
  void display() {
    cout << "hello" << endl;
  }
};
class B : public A {
 public:
  void display() {
    cout << "ciallo" << endl;
  }
};
int main() {
  B ob;
  ob.display(); // the output is ciallo
}
```

```cpp
class A {
 public:
  void display() {
    cout << "hello" << endl;
  }
};
class B : public A {
 public:
  void display(int x) {
    cout << "ciallo" << endl;
  }
};
int main() {
  B ob;
  ob.display();
}
```

```cpp
class A {
 public:
  void display() {
    cout << "hello" << endl;
  }
};
class B : public A {
 public:
  void display(int x) {
    cout << "ciallo" << endl;
  }
};
int main() {
  B ob;
  A(ob).display();
}
```

C++ Inheritance Virtual Methods

Once a method is declared as a virtual method, it becomes virtual in every subclass

Pure virtual method: Any method that is always implemented by a concrete subclass is indicated by including = 0

You can use a base class pointer to point to a derived class object, and the virtual method will run according to the type of the object pointed to by the actual pointer.

Virtual destructor

C++ Virtual Methods Practice

```cpp
class A {
 public:
  int a_;
  A() : a_(1) {}
  void display() {
    cout << a_ << endl;
  }
};
class B : public A {
 public:
  int a_, b_;
  B() : a_(2), b_(1) {}
  void display() {
    cout << a_ << " " << b_ << endl;
  }
};

class C : public B {
 public:
  int a_, b_, c_;
  C() : a_(3), c_(4) {}
  void display() {
    cout << a_ << " " << b_ << " " << c_ << endl;
  }
};

int main() {
  C oc;
  B *pb = &oc;
  pb->display();
}
```

1. Add virtual in C

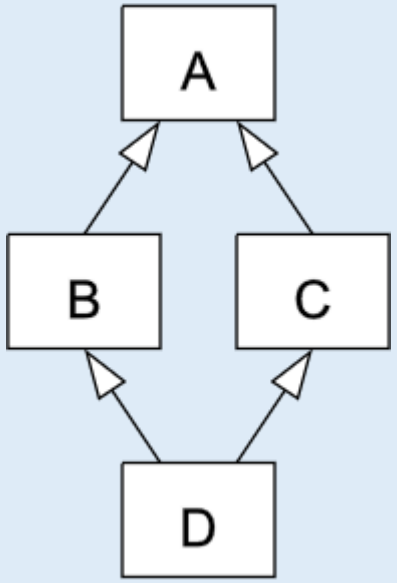
2. Add virtual in A

3. Delete int b_ in C

4. Cout << B::b_ in C

C++ Multiple Inheritance

"deadly diamond of death"




```cpp
class A {
public:
    int a = 1;
    void display() { cout << a << endl; }
};

class B: public A {
public:
    int b = 2;
    void display() {
        cout << a << b << endl;
    }
};
class C: public A {
public:
    int c = 3;
    void display() {
        cout << a << c << endl;
    }
};
class D: public B, public C {
public:
    int d = 4;
    void display() {
        cout << a << b << c << d << endl;
    }
};
```

Problem:

B, C override a method of A, which version of the method does D inherit?

A instances create twice

Virtual base class: A virtual base class allows objects derived from multiple classes (their base classes are the same) to inherit only one base class object.

If the inheritance from A to B and the inheritance from A to C are both marked virtual, C++ takes special care to only create one A object, and uses of A's members work correctly.

C++ slicing

Slicing: Derived class assigned to base class, C++ throws away any fields in the assigned object that don’t fit into the superclass by default.

To avoid slicing, one approach is to define private versions of the copy constructor and assignment operator so that copying objects in that inheritance hierarchy is prohibited

C++ template

```
template <typename ValueType>
ValueType max(ValueType v1, ValueType v2) {
   return (v1 > v2) ? v1 : v2;
}
```

The compiler will generate the code for many different versions of max, one for each type that the client uses.

Algorithm Analysis

O (bounded above) vs. Ω (below) vs. Θ (both)

Worst-case complexity
Average-case complexity

Sorting complexity	Best	Average	Worst
Selection sort	N^2	N^2	N^2
Insertion sort	N	N^2	N^2
Bubble sort	N	N^2	N^2
Merge sort	$N \log N$	$N \log N$	$N \log N$
Quicksort	$N \log N$	$N \log N$	N^2

Constant	$O(1)$	$O(Yeah)$ Finding elements in a vector using indices
Logarithmic	$O(\log N)$	$O(Nice)$ Binary search in a sorted vector
Linear	$O(N)$	$O(Fine)$ Summing a vector; linear search
$N \log N$	$O(N \log N)$	$O(K)$ Merge sort
Quadratic	$O(N^2)$	$O(Well)$ Selection sort
Cubic	$O(N^3)$	$O(My)$ Obvious algorithms for matrix multiplication
Exponential	$O(2^N)$	$O(No)$ Tower of Hanoi solution
Factorial	$O(N!)$	$O(MG)$ Brute-force search through all permutations

Data Structure

Queue
Stack
Linked List delete and insert

Maps: linear search/Binary search/hash

key--hash function->hash code collision

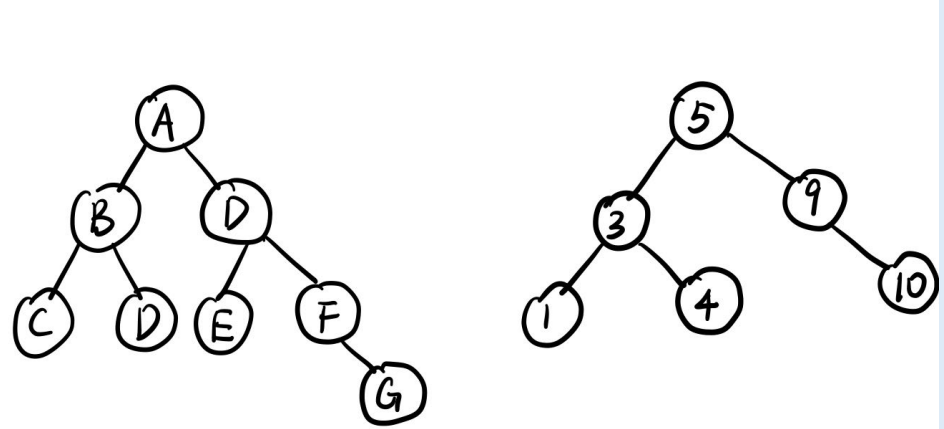
The Bucket Hashing Strategy

Open Addressing

Time complexity

Tree

Tree: root, child, parent, siblings, descendant, ancestor



Preorder traversal: 5 3 1 4 9 10
inorder traversal: 1 3 4 5 9 10
postorder traversal: 1 4 3 10 9 5
level order: 5 3 9 1 4 10

Binary Search Tree: a left subtree that contains all the nodes that come before the current node and a right subtree that contains all the nodes that come after it

A binary search tree is balanced if the height of its left and right subtrees differ by at most one and if both of those subtrees are themselves balanced.

Heap: for a node, left child 2*n+1, right child 2*n+2, parent (n-1)/2

Tree

Build a BST 5 10 8 1 2 3 4 9 13

Some numbers are inserted into BST in descending order. Which traversal method results in the same order as preorder?
A. Inorder B.postorder C.level order

C

Draw a min heap, insert order 1 2 4 5 3, delete 1 then insert 1

Set

A set is an unordered collection of distinct values.

Hash tables: insert O(1), don't support ordered iteration
Balanced binary trees: performance on the fundamental operations, but do make it possible to write an ordered iterator.
***Set* in C++ std uses Balanced binary tree and *unordered_set* (C++11) uses Hash table**

Bitwise Operators

x & *y*	Bitwise AND. The result has a 1 bit wherever both *x* and *y* have 1s. masking
x | *y*	Bitwise OR. The result has a 1 bit wherever either *x* or *y* have 1s.
x ^ *y*	Exclusive OR. The result has a 1 bit wherever *x* and *y* differ.
~*x*	Bitwise NOT. The result has a 1 bit wherever *x* has a 0.
x << *n*	Left shift. Shift the bits in *x* left *n* positions, shifting in 0s.
x >> *n*	Right shift. Shift *x* right *n* bits (logical shift if *x* is unsigned).

If x is an unsigned type, the >> operator performs a logical shift in which missing digits are always filled with 0s.
If x is a signed type, the >> operator performs what computer scientists call an arithmetic shift in which the leading bit in the value of x never changes. Thus, if the first bit is a 1, the >> operator fills in 1s; if it is a 0, those spaces are filled with 0s.

Two’s complement

0	000
1	001
2	010
3	011
−4	100
−3	101
−2	110
−1	111

```
cout << hex << -1 << ' '
<< (-1 << 1) << ' '
<< (-1 >> 1) << ' '
<< unsigned(-1) << ' '
<< (unsigned(-1) << 1) << ' '
<< (unsigned(-1) >> 1) << endl;
```

```
ffffffff fffffffe ffffffff ffffffff fffffffe 7fffffff
```

$$2^n - x \ (x > 0)$$

$$2^n - x = 1 + (2^n - 1) - x$$

Graph

A *graph* consists of a set of *nodes* together with a set of *arcs*.

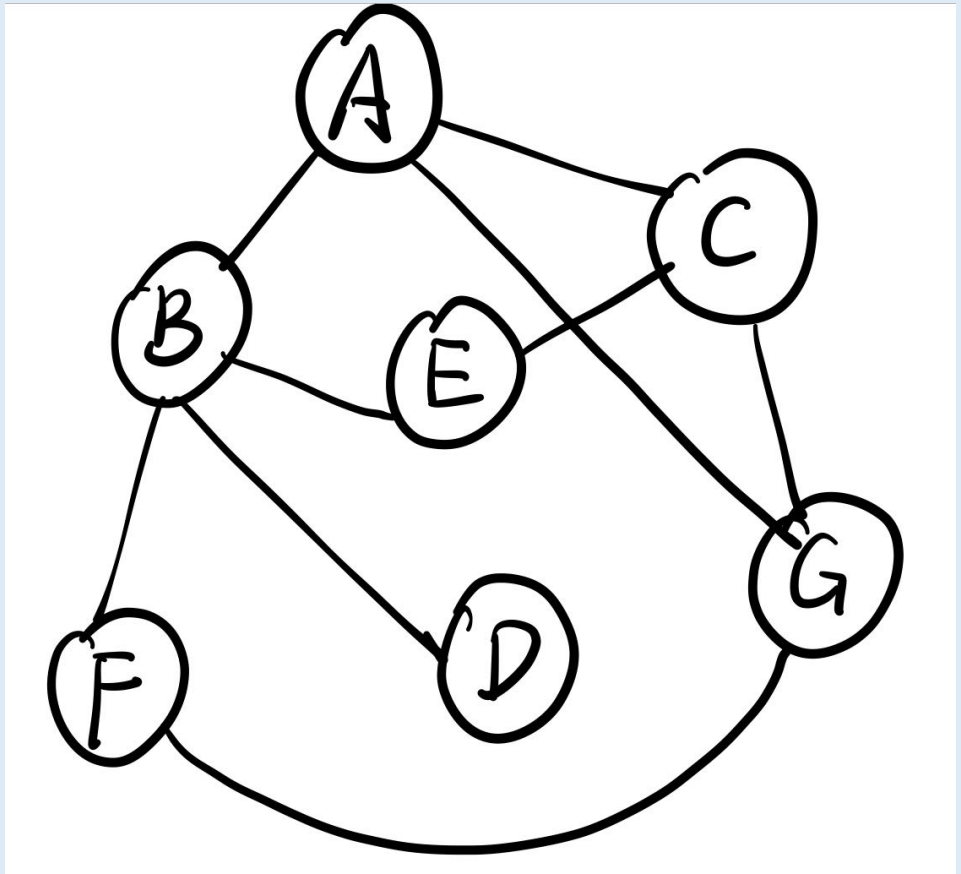
directed graphs/undirected graphs neighbors degree(in/out)
path cycle

Strongly connected: there is a path between every pair of nodes
A directed graph is only weakly connected if its undirected counterpart is connected

1.adjacency list 2.adjacency matrix 3.for each node as a *set of arcs*.

Graph DFS/BFS

Depth-first search DFS
Breadth-first search BFS



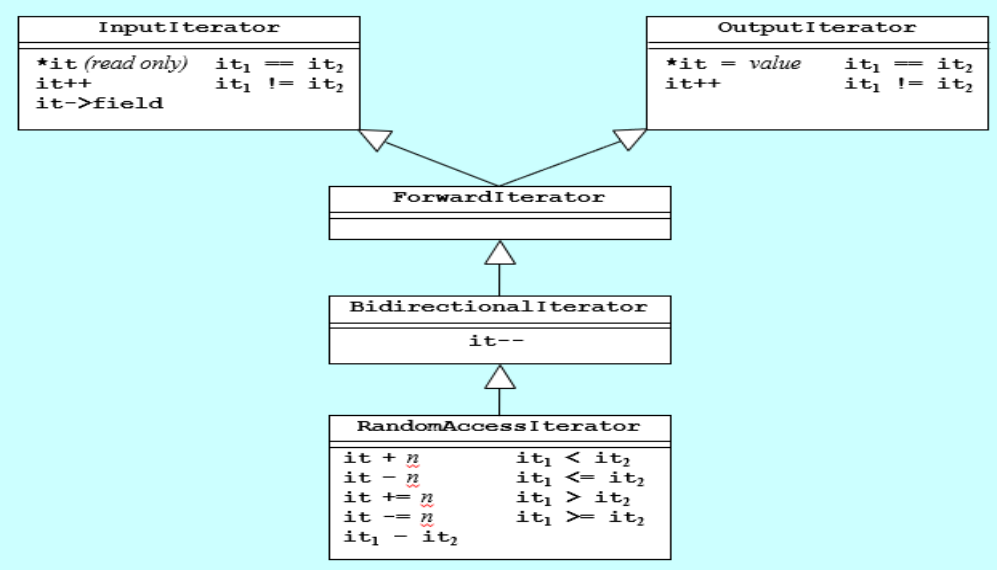
STL and Iteration

C++11 uses a range-based for statement to simplify iterators

```
for (string key : map) { . . . code to process that key . . . }
```

```
for (ctype::iterator it = c.begin(); it != c.end(); it++) {
    . . . Body of loop involving *it . . .
}
```

The purpose of iterators is to make algorithms and containers independent. Any iterator needs to implement increment and dereference, and some types need to implement decrement.



STL and Iteration

容器种类	功能
顺序容器	主要包括 vector 向量容器、list 列表容器以及 deque 双端队列容器。之所以被称为顺序容器，是因为元素在容器中的位置同元素的值无关，即容器不是排序的。将元素插入容器时，指定在什么位置，元素就会位于什么位置。
排序容器	包括 set 集合容器、multiset多重集合容器、map映射容器以及 multimap 多重映射容器。排序容器中的元素默认是由小到大排序好的，即便是插入元素，元素也会插入到适当位置。所以排序容器在查找时具有非常好的性能。
哈希容器	C++ 11 新加入 4 种关联容器，分别是 unordered_set 哈希集合、unordered_multiset 哈希多重集合、unordered_map 哈希映射以及 unordered_multimap 哈希多重映射。和排序容器不同，哈希容器中的元素是未排序的，元素的位置由哈希函数确定。

容器	对应的迭代器类型
array	随机访问迭代器
vector	随机访问迭代器
deque	随机访问迭代器
list	双向迭代器
set / multiset	双向迭代器
map / multimap	双向迭代器
forward_list	前向迭代器
unordered_map / unordered_multimap	前向迭代器
unordered_set / unordered_multiset	前向迭代器
stack	不支持迭代器
queue	不支持迭代器

$$y = \frac{1}{a} \int_0^x f(t) \sin a(x-t)\, dt$$

More about C++

Smart pointer
type cast
Exception
Template and variable parameter
STL
Rvalue reference
Move semantics
Lambda
...

Thank you!